Week 10: Advanced SQL Query System

Scenario:

The e-commerce company needs to analyze customer orders, product prices, and sales details to improve business performance. The management team wants to identify top customers, find products with higher prices, and analyze purchasing patterns. Students will use Subqueries and Nested Queries to generate advanced reports and gain useful business insights from the E-Commerce Database.

1. **Implement Subqueries and Nested Queries Concept**:
A Subquery is a query written inside another SQL query. It helps solve complex problems by using the result of one query in another query.

Types Covered:

- Single-row Subquery
- Multi-row Subquery
- Nested Subquery

2. Find Products Above Average Price Task:
The company wants to identify premium products. Students should write queries to:

1. Find products priced above the average price

```
SQL> SELECT AVG(PRICE) AS AVERAGE_PRICE FROM PRODUCT;

AVERAGE_PRICE
-------------
   16083.3333
```

2. Display product name, category, and price.

```
SQL> SELECT PRODUCT_NAME, CATEGORY_ID, PRICE FROM PRODUCT WHERE PRICE > (SELECT AVG(PRICE) FROM PRODUCT);

PRODUCT_NAME         CATEGOR      PRICE
-------------------- ------- ----------
Laptops              Cat0001      67500
Mobile               Cat0001      25000
```

3.Identify expensive products in each category.

```
SQL> SELECT PRODUCT_NAME, CATEGORY_ID, PRICE FROM PRODUCT P WHERE PRICE = (SELECT MAX(PRICE) FROM PRODUCT WHERE CATEGORY_ID = P.CATEGORY_I
D);

PRODUCT_NAME         CATEGOR      PRICE
-------------------- ------- ----------
Laptops              Cat0001      67500
TShirt               Cat0002        800
Shoes                Cat0003       1500
```

4. Find products with price higher than a selected category average

```
SQL> SELECT PRODUCT_NAME, CATEGORY_ID, PRICE FROM PRODUCT P WHERE PRICE > (SELECT AVG(PRICE) FROM PRODUCT WHERE CATEGORY_ID = P.CATEGORY_I
D);

PRODUCT_NAME         CATEGOR      PRICE
-------------------- ------- ----------
Laptops              Cat0001      67500
Mobile               Cat0001      25000
```

3. Identify Customers with Maximum Purchases Task:

Analyze customer purchasing behavior. Students should find:

1. Customer who spent the highest amount.

```
SQL> SELECT C.CUSTOMER_ID, C.CUSTOMER_NAME, SUM(O.TOTAL_AMOUNT) AS TOTAL_SPENDING FROM CUSTOMER C JOIN ORDERS O ON C.CUSTOMER_ID = O.CUSTO
MER_ID GROUP BY C.CUSTOMER_ID,C.CUSTOMER_NAME HAVING SUM(O.TOTAL_AMOUNT) = (SELECT MAX(TOTAL_SPENDING) FROM (SELECT SUM(TOTAL_AMOUNT) AS T
OTAL_SPENDING FROM ORDERS GROUP BY CUSTOMER_ID));

CUSTOMER_ID CUSTOMER_NAME                                      TOTAL_SPENDING
----------- -------------------------------------------------- --------------
       1001 Arun                                                    67500
```

2. Customers with maximum number of orders.

```
SQL> SELECT C.CUSTOMER_ID, C.CUSTOMER_NAME, COUNT(O.ORDER_ID) AS TOTAL_ORDERS FROM CUSTOMER C JOIN ORDERS O ON C.CUSTOMER_ID = O.CUSTOMER_
ID GROUP BY C.CUSTOMER_ID,C.CUSTOMER_NAME HAVING COUNT(O.ORDER_ID) = (SELECT MAX(ORDER_COUNT) FROM (SELECT COUNT(*) AS ORDER_COUNT FROM OR
DERS GROUP BY CUSTOMER_ID));

CUSTOMER_ID CUSTOMER_NAME                                      TOTAL_ORDERS
----------- -------------------------------------------------- ------------
       1001 Arun                                                         1
       1002 Priya                                                        1
       1003 Kumar                                                        1
       1004 Divya                                                        1
       1005 Rahul                                                        1
```

3. Customers whose spending is above average

```
SQL> SELECT C.CUSTOMER_ID, C.CUSTOMER_NAME, SUM(O.TOTAL_AMOUNT) AS TOTAL_SPENDING FROM CUSTOMER C JOIN ORDERS O ON C.CUSTOMER_ID = O.CUSTO
MER_ID GROUP BY C.CUSTOMER_ID,C.CUSTOMER_NAME HAVING SUM(O.TOTAL_AMOUNT) > (SELECT AVG(TOTAL_SPENDING) FROM (SELECT SUM(TOTAL_AMOUNT) AS T
OTAL_SPENDING FROM ORDERS GROUP BY CUSTOMER_ID));

CUSTOMER_ID CUSTOMER_NAME                                      TOTAL_SPENDING
----------- -------------------------------------------------- --------------
       1001 Arun                                                    67500
       1002 Priya                                                   25000
```

4.Top 5 valuable customers.

```
SQL> SELECT C.CUSTOMER_ID, C.CUSTOMER_NAME, SUM(O.TOTAL_AMOUNT) AS TOTAL_SPENDING FROM CUSTOMER C JOIN ORDERS O ON C.CUSTOMER_ID = O.CUSTO
MER_ID GROUP BY C.CUSTOMER_ID,C.CUSTOMER_NAME ORDER BY TOTAL_SPENDING DESC FETCH FIRST 5 ROWS ONLY;

CUSTOMER_ID CUSTOMER_NAME                                          TOTAL_SPENDING
----------- -------------------------------------------------- --------------
       1001 Arun                                                        67500
       1002 Priya                                                       25000
       1005 Rahul                                                        8000
       1004 Divya                                                        4200
       1003 Kumar                                                        1500
```

4. Generate Complex Business Queries

Students should create advanced SQL queries for business analysis.

Query 1: Best-Selling Product

Find products with maximum sales quantity. Tasks:

- Display product name.
- Total quantity sold.
- Total revenue generated

```
SQL> SELECT P.PRODUCT_NAME, SUM(OD.QUANTITY) AS TOTAL_QUANTITY_SOLD, SUM(OD.QUANTITY * OD.PRICE) AS TOTAL_REVENUE FROM PRODUCT P JOIN ORDE
R_DETAILS OD ON P.PRODUCT_ID = OD.PRODUCT_ID GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME HAVING SUM(OD.QUANTITY) = (SELECT MAX(TOTAL_QTY) FROM (
SELECT SUM(QUANTITY) AS TOTAL_QTY FROM ORDER_DETAILS GROUP BY PRODUCT_ID));

PRODUCT_NAME        TOTAL_QUANTITY_SOLD TOTAL_REVENUE
------------------- ------------------- -------------
TShirt                                2          1500
Shoes                                 2          4200
```

Query 2: High-Value Customers

Find customers whose total purchase amount is greater than average customer spending.

```
SQL> SELECT C.CUSTOMER_ID, C.CUSTOMER_NAME, SUM(O.TOTAL_AMOUNT) AS TOTAL_SPENDING FROM CUSTOMER C JOIN ORDERS O ON C.CUSTOMER_ID = O.CUSTO
MER_ID GROUP BY C.CUSTOMER_ID,C.CUSTOMER_NAME HAVING SUM(O.TOTAL_AMOUNT) > (SELECT AVG(TOTAL_SPENDING) FROM (SELECT SUM(TOTAL_AMOUNT) AS T
OTAL_SPENDING FROM ORDERS GROUP BY CUSTOMER_ID));

CUSTOMER_ID CUSTOMER_NAME                                          TOTAL_SPENDING
----------- -------------------------------------------------- --------------
       1001 Arun                                                        67500
       1002 Priya                                                       25000
```

Query 3: Category Performance Analysis Find:

- Highest revenue-generating category

```
SQL> SELECT P.CATEGORY_ID, SUM(OD.QUANTITY * OD.PRICE) AS TOTAL_REVENUE FROM PRODUCT P JOIN ORDER_DETAILS OD ON P.PRODUCT_ID = OD.PRODUCT_
ID GROUP BY P.CATEGORY_ID HAVING SUM(OD.QUANTITY * OD.PRICE) = (SELECT MAX(TOTAL_REVENUE) FROM (SELECT SUM(OD2.QUANTITY * OD2.PRICE) AS TO
TAL_REVENUE FROM ORDER_DETAILS OD2 JOIN PRODUCT P2 ON OD2.PRODUCT_ID = P2.PRODUCT_ID GROUP BY P2.CATEGORY_ID));

CATEGOR TOTAL_REVENUE
------- -------------
Cat0001        100500
```

- Number of products sold in each category.

```
SQL> SELECT P.CATEGORY_ID, COUNT(DISTINCT P.PRODUCT_ID) AS TOTAL_PRODUCTS_SOLD, SUM(OD.QUANTITY) AS TOTAL_QUANTITY_SOLD FROM PRODUCT P JOI
N ORDER_DETAILS OD ON P.PRODUCT_ID = OD.PRODUCT_ID GROUP BY P.CATEGORY_ID;

CATEGOR TOTAL_PRODUCTS_SOLD TOTAL_QUANTITY_SOLD
------- ------------------- -------------------
Cat0001                   3                   3
Cat0002                   1                   2
Cat0003                   1                   2
```

- Average product price by category

```
SQL> SELECT CATEGORY_ID, AVG(PRICE) AS AVERAGE_PRICE FROM PRODUCT GROUP BY CATEGORY_ID;

CATEGOR AVERAGE_PRICE
------- -------------
Cat0001         23550
Cat0002           800
Cat0003          1500
```

Query 4: Customer Purchase History Display:

- Customer name
- Number of orders
- Total purchase amount
- Most purchased product

```
SQL> SELECT C.CUSTOMER_NAME, (SELECT COUNT(*) FROM ORDERS O WHERE O.CUSTOMER_ID=C.CUSTOMER_ID) AS TOTAL_ORDERS, (SELECT NVL(SUM(O.TOTAL_AM
OUNT),0) FROM ORDERS O WHERE O.CUSTOMER_ID=C.CUSTOMER_ID) AS TOTAL_PURCHASE_AMOUNT, (SELECT PRODUCT_NAME FROM (SELECT P.PRODUCT_NAME, SUM(
OD.QUANTITY) AS QTY FROM ORDERS O JOIN ORDER_DETAILS OD ON O.ORDER_ID=OD.ORDER_ID JOIN PRODUCT P ON P.PRODUCT_ID=OD.PRODUCT_ID WHERE O.CUS
TOMER_ID=C.CUSTOMER_ID GROUP BY P.PRODUCT_NAME ORDER BY SUM(OD.QUANTITY) DESC) WHERE ROWNUM=1) AS MOST_PURCHASED_PRODUCT FROM CUSTOMER C;

CUSTOMER_NAME                                      TOTAL_ORDERS
-------------------------------------------------- ------------
TOTAL_PURCHASE_AMOUNT MOST_PURCHASED_PRODU
--------------------- --------------------
Arun                                                          1
                67500 Laptops

Priya                                                         1
                25000 Mobile

Kumar                                                         1
                 1500 TShirt


CUSTOMER_NAME                                      TOTAL_ORDERS
-------------------------------------------------- ------------
TOTAL_PURCHASE_AMOUNT MOST_PURCHASED_PRODU
--------------------- --------------------
Divya                                                         1
                4200 Shoes

Rahul                                                         1
                8000 Keyboard

Rohan                                                         0
                   0


6 rows selected.
```

5. Advanced SQL Report

Report 1: Premium Product Report

Display:

- Product Name
- Category
- Price
- Stock Availability Condition: Products above average price

```
SQL> SELECT PRODUCT_NAME, CATEGORY_ID AS CATEGORY, PRICE, STOCK_QUANTITY AS STOCK_AVAILABILITY FROM PRODUCT WHERE
 PRICE > (SELECT AVG(PRICE) FROM PRODUCT);

PRODUCT_NAME         CATEGOR      PRICE STOCK_AVAILABILITY
-------------------- ------- ---------- ------------------
Laptops              Cat0001      67500                  2
Mobile               Cat0001      25000                 10
```

Report 2: Customer Value Report

Display:

- Customer Name
- Total Orders
- Total Spending
- Customer Category

```
SQL> SELECT C.CUSTOMER_NAME, COUNT(O.ORDER_ID) AS TOTAL_ORDERS, NVL(SUM(O.TOTAL_AMOUNT),0) AS TOTAL_SPENDING, CASE WHEN NVL(SUM(O.TOTAL_AMOUNT),0)>=50000 TH
EN 'PREMIUM' WHEN NVL(SUM(O.TOTAL_AMOUNT),0)>=10000 THEN 'REGULAR' ELSE 'BASIC' END AS CUSTOMER_CATEGORY FROM CUSTOMER C LEFT JOIN ORDERS O ON C.CUSTOMER_ID
=O.CUSTOMER_ID GROUP BY C.CUSTOMER_ID,C.CUSTOMER_NAME ORDER BY TOTAL_SPENDING DESC;

CUSTOMER_NAME                                      TOTAL_ORDERS TOTAL_SPENDING
-------------------------------------------------- ------------ --------------
CUSTOME
-------
Arun                                                          1          67500
PREMIUM

Priya                                                         1          25000
REGULAR

Rahul                                                         1           8000
BASIC


CUSTOMER_NAME                                      TOTAL_ORDERS TOTAL_SPENDING
-------------------------------------------------- ------------ --------------
CUSTOME
-------
Divya                                                         1           4200
BASIC

Kumar                                                         1           1500
BASIC

Rohan                                                         0              0
BASIC


6 rows selected.
```

Report 3: Sales Performance Report

Display:

- Product-wise sales

```
SQL> SELECT P.PRODUCT_NAME, SUM(OD.QUANTITY) AS TOTAL_QUANTITY_SOLD, SUM(OD.QUANTITY*OD.PRICE) A
S TOTAL_SALES FROM PRODUCT P JOIN ORDER_DETAILS OD ON P.PRODUCT_ID=OD.PRODUCT_ID GROUP BY P.PROD
UCT_ID,P.PRODUCT_NAME ORDER BY TOTAL_SALES DESC;

PRODUCT_NAME         TOTAL_QUANTITY_SOLD TOTAL_SALES
-------------------- ------------------- -----------
Laptops                                1       67500
Mobile                                 1       25000
Keyboard                               1        8000
Shoes                                  2        4200
TShirt                                 2        1500
```

- Category-wise revenue

```
SQL> SELECT P.CATEGORY_ID, SUM(OD.QUANTITY*OD.PRICE) AS TOTAL_REVENUE FROM PRODUCT P JOIN ORDER_
DETAILS OD ON P.PRODUCT_ID=OD.PRODUCT_ID GROUP BY P.CATEGORY_ID ORDER BY TOTAL_REVENUE DESC;

CATEGOR TOTAL_REVENUE
------- -------------
Cat0001        100500
Cat0003          4200
Cat0002          1500
```

- Top-selling products

```
SQL> SELECT P.PRODUCT_NAME, SUM(OD.QUANTITY) AS TOTAL_QUANTITY_SOLD FROM PRODUCT P JOIN ORDER_DE
TAILS OD ON P.PRODUCT_ID=OD.PRODUCT_ID GROUP BY P.PRODUCT_ID,P.PRODUCT_NAME HAVING SUM(OD.QUANTI
TY)=(SELECT MAX(TOTAL_QTY) FROM (SELECT SUM(QUANTITY) AS TOTAL_QTY FROM ORDER_DETAILS GROUP BY P
RODUCT_ID));

PRODUCT_NAME         TOTAL_QUANTITY_SOLD
-------------------- -------------------
TShirt                                 2
Shoes                                  2
```

Report 4: Business Decision Report

Identify:

- High-performing products

```
SQL> SELECT P.PRODUCT_NAME, SUM(OD.QUANTITY) AS TOTAL_SOLD, SUM(OD.QUANTITY*OD.PRICE) AS TOTAL_R
EVENUE FROM PRODUCT P JOIN ORDER_DETAILS OD ON P.PRODUCT_ID=OD.PRODUCT_ID GROUP BY P.PRODUCT_ID,
P.PRODUCT_NAME HAVING SUM(OD.QUANTITY)=(SELECT MAX(TOTAL_QTY) FROM (SELECT SUM(QUANTITY) AS TOTA
L_QTY FROM ORDER_DETAILS GROUP BY PRODUCT_ID));

PRODUCT_NAME         TOTAL_SOLD TOTAL_REVENUE
-------------------- ---------- -------------
TShirt                        2          1500
Shoes                         2          4200
```

- Low-performing products

```
SQL> SELECT P.PRODUCT_NAME, NVL(SUM(OD.QUANTITY),0) AS TOTAL_SOLD FROM PRODUCT P LEFT JOIN ORDER
_DETAILS OD ON P.PRODUCT_ID=OD.PRODUCT_ID GROUP BY P.PRODUCT_ID,P.PRODUCT_NAME HAVING NVL(SUM(OD
.QUANTITY),0)=(SELECT MIN(TOTAL_QTY) FROM (SELECT P2.PRODUCT_ID, NVL(SUM(OD2.QUANTITY),0) AS TOT
AL_QTY FROM PRODUCT P2 LEFT JOIN ORDER_DETAILS OD2 ON P2.PRODUCT_ID=OD2.PRODUCT_ID GROUP BY P2.P
RODUCT_ID));

PRODUCT_NAME         TOTAL_SOLD
-------------------- ----------
Mouse                         0
```

- High-value customers

```
SQL> SELECT C.CUSTOMER_NAME, SUM(O.TOTAL_AMOUNT) AS TOTAL_SPENDING FROM CUSTOMER C JOIN ORDERS O
 ON C.CUSTOMER_ID=O.CUSTOMER_ID GROUP BY C.CUSTOMER_ID,C.CUSTOMER_NAME HAVING SUM(O.TOTAL_AMOUNT
)>(SELECT AVG(TOTAL_SPENDING) FROM (SELECT SUM(TOTAL_AMOUNT) AS TOTAL_SPENDING FROM ORDERS GROUP
 BY CUSTOMER_ID));

CUSTOMER_NAME                                      TOTAL_SPENDING
-------------------------------------------------- --------------
Arun                                                        67500
Priya                                                       25000
```

- Revenue trends

```
SQL> SELECT TO_CHAR(ORDER_DATE,'MON-YYYY') AS REVENUE_MONTH, SUM(TOTAL_AMOUNT) AS TOTAL_REVENUE
FROM ORDERS GROUP BY TO_CHAR(ORDER_DATE,'MON-YYYY') ORDER BY MIN(ORDER_DATE);

REVENUE_MONTH     TOTAL_REVENUE
----------------- -------------
SEP-2026                106200
```